

# anvil: Composable Code Transformations for R

by Sebastian Fischer, Daniel Falbel, Thomas Kalinowski, Niko German, Martin Binder, and Bernd Bischl

**Abstract** The **anvil** package offers a composable code transformation framework that allows users to express numerical algorithms efficiently in pure R. Execution is handled via just in time (jit) compilation. All code transformations are based on first tracing R code into an intermediate representation which is a computational graph. This allows combining jit-compilable operations with standard R code, the latter being “staged out” during tracing. Reverse mode automatic differentiation, which is currently the only non-lowering code transformation, is implemented as a rewriting of the computational graph into another computational graph. The package comes with two backends for jit-compilation that transform the intermediate representation into an executable: an OpenXLA backend with support for accelerators such as GPUs, as well as an experimental Fortran backend based on the **quicker** package.

## 1 Introduction

All statistical and machine learning methods fundamentally rely on some form of vector or array operations. Even though R as an interpreted language has performance limitations, relying on vectorized operations can be fast enough for many use cases. However, there are cases where the performance is not sufficient and one needs to resort to compiled languages such as C++. For example, pure R implementations might be too slow for large datasets or when no vectorized operation is available, as R loops can be prohibitively slow. Furthermore, one might additionally want to have access to GPU acceleration or automatic differentiation (AD).

### Contribution

To address this, we introduce **anvil**, which is a composable code transformation framework for the R language. Its primary goal is to offer a set of primitive operations for arrays that can be just in time (jit) compiled, differentiated, and executed on different hardware backends. It therefore allows users to implement certain numerical algorithms efficiently in pure R. The primary execution backend for **anvil** is OpenXLA (“Open Accelerated Linear Algebra”) (OpenXLA Team, 2026). It is a compiler ecosystem used across different machine learning (ML) and array computing frameworks. Components from XLA that we are using are the StableHLO intermediate representation (StableHLO Team, 2026), which is a strongly typed language for representing array programs, and PJRT (“Pluggable JIT RunTime”) (PJRT Team, 2026), which is a standardized interface for array computing frameworks with a focus on cross-platform execution. Part of our contribution are the **stablehlo** and **pjrt** R packages. The former allows users to create StableHLO programs in R, while the latter interfaces the PJRT C library and handles compilation, data allocation, and execution. There is also an experimental, alternative backend that transpiles R to Fortran code via the **quicker** package. Because the **anvil** package is primarily written in R, it is easy to extend and contribute to.

### 1.1 Related work

The biggest inspiration for **anvil** is the JAX python library, from which various design elements are adopted (Bradbury et al., 2018). Most importantly, are also using XLA as the backend and are following JAX’s functional programming style which should feel familiar to R users. Both JAX and **anvil** translate code into a computational graph by recording an evaluation trace, an approach that is well established in the AD literature (Griewank and Walther (2008), Section 1). Other features such as the handling of nested data structures or the type promotion rules are adopted from JAX, as their design is well suited for combination

with the XLA compiler. We also want to highlight the “Autodidax” tutorial, which describes the core ideas underpinning JAX ([The JAX Authors, 2024](#)), as well as `microjax`, which is a minimalistic Python library explaining JAX’s code transformation approach ([The MicroJAX Authors, 2024](#)). For the design of the `stablehlo` package and the tracing implementation in `anvil`, we have also followed a similar design as developed in `mlr3torch`, which offers a similar graph-building interface for creating neural networks. The API functions for the array operations are partly adopted from NumPy, which is a popular library for numerical computing in Python ([Harris et al., 2020](#)). For the design of the `pjrt` package, we have partly borrowed from the `gopjrt` library ([Pfeifer, 2026](#)) which offers a Go ([Team, 2025](#)) interface to PJRT.

However, `anvil` also deviates from JAX and NumPy in various ways. One important difference is that the broadcasting rules employed by `anvil` are stricter and only broadcast scalars. This is intended to avoid subtle bugs and favor explicitness over seeming convenience from NumPy’s automatic broadcasting behavior. Also, in contrast to JAX, `anvil` does not offer an eager execution mode, but instead only supports jit-compilation. The idea behind this is that for a given backend, there is a single execution model, thereby eliminating subtle bugs that arise from switching between the two modes. Instead, `anvil` offers a debug mode that performs type checking to help users identify issues with their code before compilation. Besides these user facing differences, the internal implementation of the code transformation engine is different and arguably easier to understand. Specifically, when composing different transformations, they essentially operate independently of each other, whereas in JAX, the transformations are nested, which we consider to be more difficult to reason about.

Due to being a much younger project, `anvil` also has fewer features than established numerical computing frameworks. For example the array API is not as extensive as one of the more mature array computing frameworks. Furthermore, it does not yet support forward mode AD, higher order derivatives, or batching transformations, all of which are supported by ([Bradbury et al., 2018](#)). As the project matures, we hope to close the feature gap but also anticipate more differences to emerge in order to adapt the project to the specific needs of the R community.

The main motivation behind developing `anvil` is to go beyond the limitations of the `torch` package ([Falbel and Luraschi, 2025](#)), which is a deep learning framework for the R language. It is based on LibTorch, which is the C++ library underpinning PyTorch ([Paszke et al., 2019](#)). The `torch` package offers a collection of highly optimized array operations, as well as various modules for building and training neural networks. As such, it also has support for reverse-mode AD and can be run on different hardware backends, including CPU and GPU. Thus, its general capabilities are similar to those of `anvil`, although it is more mature and contains more features. However, `torch` is primarily supports eager execution. For example, the forward function of a neural network is simply an R function that repeatedly calls into LibTorch kernels; i.e., there is no compilation step. While this has advantages regarding usability and expressiveness, it comes with performance implications. For example, all the intermediate values need to be cleaned up by the R garbage collector, even if the data itself lives on a GPU. For large neural networks that are in the primary focus of modern deep learning toolboxes, these performance implications might be less problematic, as the runtime is dominated by individual kernels such as the attention mechanism that are highly optimized. However, jit compilation can be especially attractive when working with smaller models or data, which is not unusual for statistical models. While `torch` offers a way to JIT-compile code via tracing code and converting it to TorchScript, this feature is in maintenance mode. Furthermore, it is primarily designed for deployment and has, for example, no support for automatic differentiation.

Besides `torch`, other R packages exist that offer GPU acceleration and automatic differentiation. These are `tensorflow` and `keras3`, two deep learning frameworks that interface the TensorFlow Python runtime via `reticulate`. It is also possible to use Python’s JAX library via `reticulate` ([Bradbury et al., 2018](#)). All of these packages also rely on OpenXLA for execution. While they provide access to more mature frameworks than `anvil`, they require managing

the Python dependency, as well as type conversions between R and Python. Furthermore, with **anvil** our goal is to develop an array computing library that is and will be developed primarily with the needs of the R community in mind.

Beyond those popular deep learning libraries, other R packages exist that offer hardware acceleration, automatic differentiation, jit compilation, or a combination of these. We primarily focus on those packages that are either available from standard repositories such as CRAN or Bioconductor, or which seem to be ready to be used.

A recent package that enables JIT compilation in R is the **quicker** package, which transpiles R code into Fortran code and then compiles it (Kalinowski, 2026). There are various differences between **anvil** and **quicker**. First, **quicker** has no support for automatic differentiation and currently only runs on CPU. Besides, **quicker** does not establish a new API, but instead aims to be a drop-in replacement for a set of R functions. This works because it is implemented via a static analysis approach, so the package does not need control over the functions that can be compiled, as is the case for the tracing approach followed in **anvil**. This also means that **quicker** does not allow mixing “non-compilable” R code with compilable code, as is possible in **anvil**, where the “non-compilable” code is staged out during tracing to the intermediate representation. The tracing approach is more flexible, but also requires the user to be more careful about the code that they are writing. One notable limitation of the tracing approach is that it does not support recursive function calls. There is also the **ast2ast** package, which translates a subset of R code into C++ (Konrad Krämer, 2026). It is similar in spirit to **quicker**, but also supports forward and backward-mode automatic differentiation. In contrast to **anvil**, it does not support execution on accelerators like GPUs.

Then, there are **odin** and **rxode2**, both of which focus on differential equations and compile a small subset of R code via translation to C, similar to **quicker**. However, they do not support GPU acceleration and their set of supported operations is more limited than **anvil**’s.

Another category of packages offer special DSLs for statistical models which are compiled. There is the **brms** package (Bürkner, 2017) which allows users to define Bayesian models via the familiar formula interface and translates them into Stan models which are compiled (Carpenter et al., 2017). Stan itself is a probabilistic programming language and also has GPU support via OpenCL (Munshi, 2009). It is also possible to directly use Stan in R, e.g. via the **rstan** package, but this requires writing the program itself in Stan. There also exists **nimble**, which is primarily intended for fitting hierarchical statistical models and works by generating C++ code and compiling it. It also supports compiling custom R functions by generating C++ code, as well as forward and backward-mode automatic differentiation via CppAD (Bell, 2012). However, there is no support for GPU acceleration.

Then, there is also the **compiler** package of the R language itself (R Core Team, 2025). It is used by the R interpreter to compile arbitrary R code into byte-code that is executed by the R virtual machine. However, because it has to handle arbitrary R code, the possible optimizations are limited due to R’s design (Morandat et al., 2012). This generally highlights the trade-off between expressiveness and performance. In **anvil**, we are focusing on a rather narrow set of numerical operations, which thus allows for aggressive optimizations and highly efficient execution.

While various R packages offer GPU acceleration for specific operations (e.g., **xgboost**), generic GPU-accelerated array computing frameworks beyond the deep learning libraries from earlier are rare. One noteworthy mention is **OpenCL**, which offers a low-level interface to OpenCL.

Besides the packages already listed above, other R libraries exist with automatic differentiation capabilities. For example, forward mode automatic differentiation via dual numbers is supported by the packages **nabla**, **salad**, **dual**, and **madness** (Towell, 2026, Perdry (2024), Sartore (2025), Pav (2023)). None of these packages have JIT compilation capabilities or support GPU execution.

There is also the **RTMB** package which allows for forward and backward-mode automatic differentiation and interfaces the TMB C++ library ((Kristensen et al., 2016)).

Despite many packages offering some subset of **anvil**'s features, none of the native R packages simultaneously offer GPU acceleration, JIT compilation, and backward-mode AD.

The structure of the paper is as follows: First, we will briefly review some important background information, including the OpenXLA compiler, the **quickr**, and some basics on automatic differentiation. In the subsequent part, we will describe the main design of the **anvil** package. Then, we will demonstrate the package's abilities, using logistic regression as a simple example. Afterwards, we will show how it is possible to extend the package. Then, we will include some small CPU runtime benchmarks to illustrate the performance of the package. Finally, we will conclude by discussing current limitations and directions for future work.

Besides other software libraries, there is of course a considerable amount of mathematical literature on automatic differentiation. The literature is far too vast to cover appropriately here. One notable mention is the book by Griewank and Walther ([Griewank and Walther, 2008](#)), which covers the topic in depth. Beyond that, there are various papers that describe specific algorithms for more complex operations such as Cholesky decomposition that we used for the implementation of the reverse-mode AD rules.

## 2 Background

### 2.1 The OpenXLA Compiler Ecosystem

The OpenXLA project consists of various individual components, but we will only cover the StableHLO intermediate language (IR) and the PJRT library which are used directly in **anvil**. One of the main goals of the OpenXLA project is to establish standardized interfaces between numerical computing frameworks such as JAX or **anvil** and their underlying execution engines. This is achieved via the following two components:

1. The "Pluggable Jit RunTime" (PJRT) which defines a common interface for execution engines ([PJRT Team, 2026](#)).
2. The StableHLO language, which is the intermediate representation that produced by the framework and consumed by the backend ([StableHLO Team, 2026](#)).

If a numerical computing framework produces StableHLO programs and handles the execution via PJRT, it can be run on any hardware for which a PJRT plugin exists. This turns an  $n \times m$  problem ( $n$  frontends need to implement their execution for  $m$  different hardware backends) into a  $n + m$  problem.

#### StableHLO

An example for the StableHLO intermediate representation is given below. The main program takes in two f32 arrays of shape 2x2, multiplies them, and returns the result.

```
func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) -> tensor<2x2xf32> {
  %0 = stablehlo.multiply %x, %y : tensor<2x2xf32>
  return %0 : tensor<2x2xf32>
}
```

It is expressed in MLIR (Multi-Level Intermediate Representation). MLIR is part of the LLVM compiler infrastructure and provides a framework to define IRs at multiple levels, also called dialects, that are transformed into each other via so-called passes. Such passes can, e.g., perform optimizations or lower the abstraction level. It is in single static assignment (SSA) form, meaning that each variable is assigned to exactly once, and is fully functional. In order to use the StableHLO language in **anvil**, it needs to be possible to translate R code into StableHLO programs. For this purpose, we have created the **stablehlo** package. The package essentially consists of four parts:

1. An S3 class representation of the StableHLO language, which is similar to the StableHLO specification.
2. A `repr()` generic that converts an object into its StableHLO string representation that can be consumed by the XLA compiler.
3. A set of type inference functions for the primitive operations.
4. A builder API for easily creating StableHLO programs.

The package is implemented fully in R, instead of interfacing the C++ implementation, which simplifies the installation. In order to stay close to the language definition, the `stablehlo` package uses 0-based indexing for dimensions and other indices. Note that the `stablehlo` package is only a partial implementation of the language and does not support all features. For example, there is no support for complex numbers, various primitive operations are missing, and there is currently no support for unknown dimensions, which is commonly referred to as shape dynamism. Nonetheless, the many important features and primitive operations are already supported.

Next, we will illustrate how to use the builder API to create the StableHLO program from above. Below, we create a new StableHLO function with the name “main” and register two input parameters with the names “x” and “y” and specify their type.

```
library(stablehlo)
func <- local_func("main")
arg1 <- hlo_input("x", "f32", shape = c(2, 2), func = func)
arg2 <- hlo_input("y", "f32", shape = c(2, 2), func = func)
out <- hlo_multiply(arg1, arg2)
f <- hlo_return(out)
hlo_string <- repr(f)
```

The builder API works via reference semantics: `local_func()` creates a `Func` object representing the program being built, and subsequent calls to `hlo_input()`, `hlo_multiply()`, modify it in-place. Their return values are `FuncValues`, which represent a specific value in a StableHLO program and also contain a reference to the `Func`. Internally, each `hlo_<op>` operation (like `hlo_multiply()`) expect all the input `FuncValues` to be part of the same `Func`. Then, a type checking function is run `infer_types_multiply` that computes the output type for a valid combination of inputs. Afterwards, a new operation is appended to the underlying `Func` and a `FuncValue` representing the result of the multiplication is returned. Finally, `hlo_return()` marks the outputs, and `repr()` converts the `Func` to a string:

```
cat(hlo_string)

#> func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) -> tensor<2x2xf32> {
#> %0 = stablehlo.multiply %x, %y : tensor<2x2xf32>
#> return %0 : tensor<2x2xf32>
#> }
```

While some details were omitted in the above example, this shows the core ability of the `stablehlo` package to easily create StableHLO programs in R. For some more details, see the StableHLO specification<sup>1</sup> or the [this vignette](#).

## PJRT

The `pjrt` package is the interface to the PJRT C++ library, implemented using `Rcpp` (Edelbuettel, 2013). The PJRT library itself is header-only and only defines the interface for the underlying execution engine. A specific implementation of the interface (e.g. for CPU or NVIDIA GPU) is accessed by downloading prebuilt shared libraries and loading them

<sup>1</sup><https://openxla.org/stablehlo/spec>



dynamically. Currently, these prebuilt PJRT plugins are obtained from the zig machine learning framework (zml). Core components of the PJRT interface are buffer allocation, program compilation, and program execution, but many other API endpoints exist. A key design principle of the `pjrt` package is asynchronous dispatch: operations like buffer creation and program execution return immediately, allowing R to continue preparing work while the device computes in the background. This enables efficient overlap of host and device work, which is important for performance on accelerators but also when utilizing many CPU cores. Just like for the `stablehlo` package, we only show the core functionality of `pjrt` here and refer to the package website<sup>2</sup>.

As a first step, we will compile the StableHLO program from above. For this, the `character(1)` first needs to be converted into a `PJRTProgram` object.

```
library(pjrt)
program <- pjrt_program(src = hlo_string, format = "mlir")
```

Then, this `PJRTProgram` can be compiled into a `PJRTLoadedExecutable` object. For this, a specific `PJRTClient` object is needed, some optional compile options, as well as a device specification. The `PJRTClient` contains a `PJRTPlugin`, which is a dynamic library that implements the PJRT interface for a specific hardware backend. We can create a `PJRTClient` object by passing a string identifier such as `"cpu"` or `"cuda"`.

```
client <- pjrt_client("cpu")
client
```

```
#> <PJRTClient:cpu>
```

To compile the program via `pjrt_compile()`, we don't specify a client, but instead pass a device specification. For CPU, the distinction between device and client is not so important as there is usually only one CPU, so there is a one-to-one correspondence. When working with GPUs, there might be multiple GPUs available and the device represents one of them (by default the first one), and it matters on which GPU the buffer will be allocated. However, multi-GPU setups are not properly supported in `pjrt` yet, but this could be added in the future.

```
executable <- pjrt_compile(program, device = pjrt_device("cpu"))
executable
```

```
#> <PJRTLoadedExecutable>
```

In order to execute this program, `PJRTBuffer` objects need to be created that are multi-dimensional arrays. Below, we create two simple `f32` arrays of shape `2x2` and specify the device to be the CPU, which will use the buffer allocation implementation of the PJRT CPU plugin.

```
x <- pjrt_buffer(1:4, shape = c(2, 2), dtype = "f32", device = "cpu")
x
```

```
#> PJRTBuffer
#>  1 3
#>  2 4
#> [ CPUf32{2x2} ]
```

```
y <- pjrt_buffer(5:8, shape = c(2, 2), dtype = "f32", device = "cpu")
y
```

---

<sup>2</sup><https://r-xla.github.io/stablehlo/>

```
#> PJRTBuffer
#> 5 7
#> 6 8
#> [ CPUf32{2x2} ]
```

We can multiply the two buffers with each other by passing them to the `pjrt_execute()` function along with the executable, and convert back to R via `as_array()`.

```
out <- pjrt_execute(executable, x, y)
as_array(out)
```

```
#>      [,1] [,2]
#> [1,]    5   21
#> [2,]   12   32
```

Various extractors exist for PJRTBuffer objects such as `shape()` and `device()`. Because PJRTBuffer objects are external pointers, custom (de)serialization is needed; for this, we implement the safetensors format (Falbel, 2025).

## 2.2 The **quickr** Fortran Transpiler

The **quickr** package serves as a second, experimental, backend for **anvil** and was independently developed. It transpiles R code to Fortran code and compiles it. Below, we can recreate the StableHLO program from above. One semantic difference is that it uses data type `f64` instead of `f32` because **quickr** operates directly on standard R objects as opposed to **pjrt** which defines its own array type.

```
library(quickr)

multiply_r <- function(x, y) {
  declare(
    type(x = double(NA, NA)),
    type(y = double(NA, NA))
  )
  x * y
}
f <- quick(multiply_r)
f(
  matrix(c(1, 2, 3, 4), nrow = 2),
  matrix(c(5, 6, 7, 8), nrow = 2)
)

#>      [,1] [,2]
#> [1,]    5   21
#> [2,]   12   32
```

There are also various other differences with respect to the underlying implementation. Instead of offering a builder API like in **stablehlo**, **quickr** works via static analysis of the R code. Furthermore, the argument types of the inputs need to be specified via the `declare()` function. Here, `double(NA, NA)` means a 2-dimensional array with unknown dimensions, but concrete dimensions can also be specified. One advantage of **quickr** over **pjrt** and **stablehlo** is that the former does not necessitate recompilation for every new input type combination (which includes the shapes). A disadvantage is that **quickr** currently does not support GPU acceleration, supports fewer datatypes and is less optimized than the XLA backend.

### 2.3 Automatic Differentiation

Besides JIT compilation, another core feature of **anvil** is its ability to automatically compute derivatives of R functions. Here, we briefly cover some central ideas behind AD to provide the background for understanding **anvil**'s design. For a comprehensive treatment, we refer the reader to (Griewank and Walther, 2008).

#### The Jacobian

Following Griewank and Walther (2008), consider a function  $F : \mathcal{D} \subset \mathbb{R}^n \rightarrow \mathbb{R}^m$ , where  $\mathcal{D}$  is an open subset of  $\mathbb{R}^n$ . The Jacobian of  $F$  is defined as the matrix-valued function  $F' : \mathcal{D} \subset \mathbb{R}^n \rightarrow \mathbb{R}^{m \times n}$ , where the entries are given by  $F'(x)_{ij} = \frac{\partial F_i}{\partial x_j}$ .

#### Evaluation Procedures and the Chain Rule

AD exploits the fact that every computer program, no matter how complex, executes a sequence of operations whose derivatives are known. Following Griewank and Walther (2008), suppose  $F$  is evaluated by a computer program. Any such evaluation can be decomposed into a sequence of vector-valued *primitive functions*  $\varphi_i$  (called *elemental functions* in Griewank and Walther (2008)) such as addition, multiplication, sin, exp, etc. The evaluation produces a sequence of variables  $v_1, \dots, v_{n+l+m}$ , where  $v_1, \dots, v_n$  are the input variables (i.e.,  $x$ ),  $v_{n+1}, \dots, v_{n+l}$  are intermediate variables, and  $v_{n+l+1}, \dots, v_{n+l+m}$  are the output variables (i.e.,  $y$ ). Each non-input variable is obtained by applying a primitive function to some of the preceding variables:

$$v_i = \varphi_i(v_j)_{j \prec i}, \quad i = n+1, \dots, n+l+m$$

The *data dependence relation*  $j \prec i$  indicates that  $v_j$  is a direct argument of  $\varphi_i$ . This sequence of assignments is called an *evaluation procedure* and can be represented as a directed acyclic graph (DAG)  $G = (V, E)$ , where the vertex set  $V = \{v_1, \dots, v_{n+l+m}\}$  contains all input, intermediate, and output variables, and the edge set  $E = \{(v_j, v_i) : j \prec i\}$  encodes the data dependencies. The relation  $\prec$  thus defines a partial order on the vertices, and any topological ordering of  $G$  yields a valid evaluation sequence. We refer to  $G$  as the *computational graph* of  $F$ .

A function is  $d$ -times continuously differentiable ( $C^d$ ) if all its derivatives up to order  $d$  exist and are continuous. If each primitive function  $\varphi_i \in C^d(\mathcal{D}_i)$  on its open domain  $\mathcal{D}_i$ , then the composite function  $F$  inherits this smoothness:  $F \in C^d(\mathcal{D})$  on the open set of inputs for which the evaluation procedure is well-defined (Griewank and Walther (2008), Proposition 2.1). This guarantees that the chain rule can be applied systematically to compute derivatives of  $F$ , and that these derivatives are numerically stable (small input perturbations yield small changes in derivative values). Most standard primitives ( $+$ ,  $*$ , sin, exp, log) are  $C^\infty$  on their domains.

#### Jacobian-Vector Products and Vector-Jacobian Products

While computing the full Jacobian is often too expensive, two linear maps derived from it are central to AD:

- The *Jacobian-vector product* (JVP): given a tangent vector  $\dot{x} \in \mathbb{R}^n$ , the JVP computes  $F'(x)\dot{x} \in \mathbb{R}^m$ . This is a directional derivative, describing how the output changes when the input is perturbed in the direction  $\dot{x}$ .
- The *vector-Jacobian product* (VJP): given an adjoint (or cotangent) vector  $\bar{y} \in \mathbb{R}^m$ , the VJP computes  $\bar{y}^\top F'(x) \in \mathbb{R}^n$ . This describes how a linear functional of the output depends on each input.



A single JVP evaluation costs roughly the same as evaluating  $F$  itself, but yields only one column's worth of information about  $F'$ . Similarly, a single VJP evaluation yields one row's worth. The two main modes of AD differ in which of these products they compute, and therefore in the *order* in which the chain rule is evaluated, leading to different computational trade-offs.

### Forward Mode

Forward-mode AD propagates tangent vectors forward through the evaluation procedure: given  $\dot{x} \in \mathbb{R}^n$ , it computes  $\dot{v}_i = \sum_{j \succ i} \frac{\partial \varphi_i}{\partial v_j} \dot{v}_j$  for each variable, yielding the JVP  $\dot{y} = F'(x)\dot{x}$ . A single forward sweep costs roughly the same as evaluating  $F$ , so obtaining the full Jacobian requires  $n$  sweeps. This makes forward mode efficient when  $n \ll m$ , but costly for the common case of differentiating a scalar loss with respect to many parameters.

### Reverse Mode

Reverse-mode AD (also called the *adjoint* mode) propagates sensitivities backward through the evaluation procedure. Given an adjoint vector  $\bar{y} \in \mathbb{R}^m$ , it computes for each variable  $v_i$  the corresponding adjoint  $\bar{v}_i$  (a vector of the same dimension as  $v_i$ ):

$$\bar{v}_i = \sum_{k \succ i} \left( \frac{\partial \varphi_k}{\partial v_i} \right)^\top \bar{v}_k$$

proceeding in reverse order, from outputs to inputs, where  $k \succ i$  denotes that  $v_i$  is a direct argument of  $\varphi_k$ . Each term  $\left( \frac{\partial \varphi_k}{\partial v_i} \right)^\top \bar{v}_k$  is a local vector-Jacobian product (VJP). The final result  $\bar{x} = F'(x)^\top \bar{y}$  is the VJP of  $F$ .

The key advantage of reverse mode is that for a scalar-valued function ( $m = 1$ ), setting  $\bar{y} = 1$  yields the full gradient  $\nabla F(x) \in \mathbb{R}^n$  in a *single* reverse sweep, regardless of  $n$ . This result is sometimes referred to as the “cheap gradient principle”: the computational cost of evaluating the gradient is bounded by a small constant multiple (typically 2–5) of the cost of evaluating  $F$  itself (Griewank and Walther (2008), Chapter 3). The trade-off is that reverse mode requires access to the intermediate values  $v_i$  from the forward evaluation during the backward sweep, introducing additional memory overhead.

Since the most common use case is computing the gradient of a scalar-valued loss ( $m = 1$ ) with respect to many parameters ( $n \gg 1$ ), reverse mode is the natural choice. Currently, **anvil** implements only reverse-mode AD; support for forward mode is planned for a future release. In **anvil**, backward-mode AD is implemented as a graph-to-graph transformation, as described in the next section. Each primitive operation stores its own local VJP rule, and the transformation walks the computational graph in reverse topological order to accumulate gradients.

### Non-Differentiable Functions

However, some commonly used operations — such as  $\text{abs}$ ,  $\text{max}$ ,  $\text{min}$ , or  $\text{ReLU}$  — are not differentiable at isolated points. Griewank and Walther (2008) (Chapter 14) shows that such functions are *piecewise differentiable*: they can be represented as a selection of finitely many smooth functions, and the non-differentiable points (boundaries between selections) have measure zero. Since **anvil** operates on a fixed evaluation trace, each trace corresponds to a single smooth selection, and AD differentiates that branch. At non-differentiable points, the derivative of whichever branch was taken is returned — for example,  $\text{abs}'(0) = 0$  and  $\text{ReLU}'(0) = 0$ . This is the standard convention adopted by modern AD frameworks.

### 3 Package Design and Features

#### □ ASYNC!!

Having covered the underlying basics, we will now describe the main design of the **anvil** frontend. Beyond the **stablehlo** and **pjrt** packages introduced earlier, the ecosystem includes two supporting packages: **tengen**, which defines S3 generics for tensor operations such as `shape()`, `dtype()`, and `as_array()`, and **xlamisc**, which provides shared utilities used across the other packages. The core idea of the package is centered around code transformations, an approach adopted from JAX (Bradbury et al., 2018). In general, there are three types of transformations that are executed in sequence, with step 2 being executed an arbitrary number of times, possibly with different transformations (although we currently only support reverse-mode AD).

1. Transforming R code into a computational graph.
2. Transforming a computational graph into another computational graph (e.g. for automatic differentiation).
3. Transforming a computational graph into an executable (either via **stablehlo** and **pjrt** or **quickr**).

We will now cover these three types of transformations in more detail. Then, we explain how these transformations are wrapped in the main user interface.

#### 3.1 Code transformations

##### Creating a Computational Graph from R Code

The first step in the pipeline is to simplify the R code into a computational graph. When it comes to translating code into other code, there are in principle two approaches:

1. Static analysis of the code.
2. Recording an *evaluation trace* of the code by abstractly evaluating it.

In **anvil**, we are following the latter approach, because it allows to seamlessly combine standard R code with jit-compilable operations. The function that takes in an R function and returns a computational graph is called `trace_fn()`. In order for this to be useful, the function needs to (either directly or indirectly) call into **anvil** primitive functions, i.e., `nv1_<op>` functions. These primitive functions are similar to the stableHLO operations like `hlo_add` from earlier. In many cases, the primitive functions are not very ergonomic to use and we recommend using the thin wrappers like `nv_add` instead that additionally perform scalar broadcasting and type casting. These thin wrappers call into other **anvil** primitive functions, however. The primitive functions are essentially graph building functions, also similar to the `hlo_add` function from earlier. Below, we create an R function that takes in two values and multiplies them. Note that we could have equally used the overloaded `*` infix operator, but use `nv_mul` for clarity.

```
library(anvil)
multiply_r <- function(x, y) {
  nv_mul(x, y)
}
```

We can now trace this function into a computational graph. At the beginning, this will create an `anvil::GraphDescriptor` that primitive functions can attach to to build up the graph. The types for the inputs are inferred from the example arguments. The `trace_fn` transformation uses these example inputs to obtain abstract `GraphBox` objects, which are passed around during the evaluation of the function. Such a `GraphBox` is the core object

that allows to build up the graph. It contains a `GraphNode`, which is either a `GraphValue` or a `GraphLiteral` and a `GraphDescriptor`, which is essentially a graph in the making. The `GraphNode` contains an `AbstractTensor` (or one of its subclasses) which consists primarily of a shape and data type. Below, `nv_abstract` creates `AbstractArray` objects. Internally, a new `GraphDescriptor` is initialized and `GraphValue` and `GraphBox` objects are created from the provided `AbstractTensor`.

```
graph <- trace_fn(multiply_r, list(x = nv_abstract("f32", integer(0)), y = nv_abstract("f32", integer(0))))
graph
```

```
#> <AnvilGraph>
#>   Inputs:
#>     %x1: f32[]
#>     %x2: f32[]
#>   Body:
#>     %1: f32[] = mul(%x1, %x2)
#>   Outputs:
#>     %1: f32[]
```

Once the evaluation of `multiply_r` with the special `GraphBox` objects is complete, the `GraphDescriptor` is converted into a `Graph` object. The only difference between them is that `GraphDescriptor` does some book-keeping during program creation which is not needed anymore afterwards.

From the printed output of the graph we can see that a graph has:

1. Inputs: `GraphValues` that are the inputs to the graph.
2. Constants: `GraphValues` that are constants in the graph.
3. Body: A list of `PrimitiveCall` objects, which represent the primitive operations that make up the graph.
4. Outputs: `GraphValues` that are the outputs of the graph.
5. In-tree and Out-tree: Because Graphs only take in array values, but we want to support nested data structures, the in-tree and out-tree define how to (un-)flatten the inputs and outputs of the graph.

An important consequence of the tracing approach is that the shapes of all intermediate values must be determined at trace time. This is because the StableHLO language (and the XLA compiler) require all shapes to be statically known. In practice, this means that the traced function cannot contain operations that produce data-dependent shapes, such as `which()` or `unique()`.

### Transforming a Graph into another Graph

Once our program no longer consists arbitrary R code, but is in a standardized form, we can transform the program by creating another `Graph` object. Currently, the only example for such a type of transformation is the gradient transformation, which is implemented in the `transform_gradient()` function. It takes in a `Graph` and the names of the arguments to compute the gradient with respect to.

```
gradient_graph <- transform_gradient(graph, wrt = c("x", "y"))
```

The `anvil` package does not put any restrictions on *how* such a transformation is implemented. Usually, however, such a transformation needs to store information for how to transform each primitive. For this reason, the `anvil::Primitive` objects have a `@rules` field that can be populated. For example, the reverse rule for the addition primitive is implemented as follows:

```

prim("add")["reverse"]

#> function (inputs, outputs, grads, .required)
#> {
#>   grad <- grads[[1L]]
#>   list(if (.required[[1L]]) grad, if (.required[[2L]]) grad)
#> }
#> <bytecode: 0x1317206b8>
#> <environment: namespace:anvil>

```

More concretely, `transform_gradient()` walks the graph in reverse topological order and, for each `PrimitiveCall`, looks up the reverse rule stored in the primitive's `@rules["reverse"]` field. This rule receives the original inputs and outputs from the forward pass as well as the incoming gradient, and returns the gradient contributions for each input. These contributions are accumulated across all uses of a value to obtain the total gradient. Other transformations such as forward mode AD or batching transformations (similar to R's `Vectorize`) could be implemented similarly.

Because the reverse rules are themselves expressed in terms of primitive operations (i.e., they call `nv1_*` functions), the gradient graph is built using the same tracing mechanism as the forward pass. This means that the gradient computation is itself a Graph that can be further transformed or compiled — for example, one could compute higher-order derivatives by applying `transform_gradient()` again to the gradient graph. The reverse rules are stored per-primitive in the `@rules` field, rather than being hard-coded in the transformation, making it straightforward to add differentiation support for new primitives (see Section 2.5).

## Compiling a Graph

Whenever we are done transforming our Graph and are ready to execute it, we need to lower it. We can convert a Graph into a stablehlo program via the `stablehlo()` function. The output of this is a list containing:

1. The StableHLO program.
2. The constant buffers in the Graph.

```

out <- stablehlo(graph)
cat(repr(out[[1]]))

#> func.func @main (%0: tensor<f32>, %1: tensor<f32>) -> tensor<f32> {
#> %2 = stablehlo.multiply %0, %1 : tensor<f32>
#> return %2 : tensor<f32>
#> }

```

The output is a list containing the StableHLO program and any constant buffers. Non-scalar constants are not inlined into the program but added as additional inputs, to avoid increasing program size and compile times.

Next, we will describe how these transformations are wrapped in the main user interface.

## Broadcasting

Unlike NumPy or R's built-in recycling, `anvil`'s primitive operations (`nv1_*`) do not automatically broadcast their inputs — operands must have exactly matching shapes. This is a deliberate design choice: implicit broadcasting is a common source of subtle bugs, especially when shapes are dynamic or when operations are composed across multiple transformations.

The API-level wrappers (`nv_add`, `nv_mul`, etc.) do perform *scalar* broadcasting — if one operand is a scalar and the other is an array, the scalar is broadcast to match — but they do

not perform full NumPy-style broadcasting. For explicit shape manipulation, functions like `nv_broadcast_to()` and `nv_broadcast_arrays()` are provided:

```
a <- nv_array(1:3, "f32", shape = c(1, 3L))
b <- nv_array(1, "f32", shape = 1L)
# This would fail: nv_l_add(a, b)
# Instead, broadcast explicitly:
b_broadcast <- nv_broadcast_to(b, c(1, 3L))
nv_add(a, b_broadcast)

#> f32{1,3}
```

More generally, broadcasting could itself be implemented as a code transformation that wraps a function and automatically inserts the necessary broadcast operations before each primitive call, analogous to `broadcast(nv_add)`. This is not currently implemented, but illustrates how the code transformation approach naturally extends to concerns beyond differentiation and compilation.

### Type Promotion

When combining arrays of different data types in a binary operation, **anvil** needs to determine a common type. The type promotion rules are adopted from JAX and are specifically designed for execution on accelerators like GPUs, where one typically wants speed over precision. In particular, the default floating point type is `f32` rather than R's `f64`, because 32-bit operations are significantly faster on GPUs and sufficient for many applications.

A key aspect of the promotion system is the concept of *ambiguous types*, which correspond directly to what JAX calls *weak types*. In JAX, values created from Python scalars or literals are assigned weak types, meaning the type system treats them as having a suggested rather than a definitive type. **anvil** adopts the same concept: scalar literals such as `1L` or `3.14` receive ambiguous (weak) types, denoted `i32?` and `f32?` respectively. When an ambiguous value is combined with a non-ambiguous (strong) one, the non-ambiguous type takes precedence. For example, multiplying an `f32?` literal with an explicitly created `f64` array produces an `f64` result, whereas two `f32?` literals would produce `f32`. This prevents R literals from inadvertently widening types, which is especially important when writing code intended for GPU execution where `f64` operations can be an order of magnitude slower than `f32`.

## 3.2 Main User Interface

The main user interface is centered around `jit()` and transformations functions like `gradient()` or `value_and_gradient()`. Furthermore, we can create arrays, which we will cover first.

### The AnvilArray

In order to create data for our programs, we can use the `nv_array()`, `nv_scalar()` and `nv_empty()` functions. These essentially wrap the `pjrt_buffer()` and `pjrt_scalar()` functions from the **pjrt** package.

```
x <- nv_scalar(1, "f32")
x

#> AnvilArray
#> 1
#> [ CPUf32{} ]
```

```

y <- nv_array(1:4, "f32")
y

#> AnvilArray
#>  1
#>  2
#>  3
#>  4
#> [ CPUf32{4} ]

```

Just like the PJRTBuffer objects, AnvilArray objects can be serialized and deserialized using the [safetensors](#) package.

An important design property of AnvilArray objects is that they follow value semantics: they are conceptually immutable. While subassignment like `x[1] <- 2` is syntactically supported, it does not modify the array in place but instead creates a new array. This aligns naturally with R's copy-on-modify semantics for `[<-`, where modifying a shared object creates a copy. It also mirrors StableHLO's fully functional, single static assignment (SSA) design, where every operation produces a new value rather than mutating an existing one. This correspondence means that R's idiomatic use of `[<-` translates directly to StableHLO's functional update pattern, without requiring any special handling. The immutability ensures that functions operating on AnvilArray objects are pure, which is essential for the correctness of the code transformation approach. Within a jit-compiled function, the XLA compiler can optimize away unnecessary copies, similar to R's copy-on-write mechanism. However, when calling a jit-compiled function repeatedly from R (e.g., in a training loop), the compiler cannot optimize memory reuse across calls. For this case, the `jit()` function accepts a `donate` parameter that specifies which input buffers may be reused for the output, enabling in-place updates without violating the functional semantics:

```

update_fn <- jit(function(params, grads) {
  list(params = params - 0.01 * grads)
}, donate = "params")

```

After donation, the original buffer is invalidated and should not be used again.

### **jit()**

`jit()` is the main function that takes in an R function and returns a JIT-compiled function.

```

multiply_print_r <- function(x, y) {
  print("Hello useR!")
  nv_mul(x, y)
}
multiply_jit <- jit(multiply_print_r)
multiply_jit(x, y)

#> [1] "Hello useR!"

#> AnvilArray
#>  1
#>  2
#>  3
#>  4
#> [ CPUf32{4} ]

```

The returned function contains the “recipe” for transforming the R function into a Graph, then lowering it into a StableHLO program and finally executing it. Compiling the



function on every call would be inefficient, so a cache stores the executable for each unique combination of input types (i.e., shapes and data types). Subsequent calls with the same input types reuse the cached executable, while calls with new types trigger recompilation. Because only the primitive operations are recorded in the graph, any standard R code (such as `print()` statements) is executed during tracing but not during subsequent cached executions.

JIT-compiled functions can also take in and return lists of arrays.

```
multiply_jit_nested <- jit(function(lst) {
  list(nv_mul(lst[[1]], lst[[2]]))
})
multiply_jit_nested(list(x, y))

#> [[1]]
#> AnvilArray
#> 1
#> 2
#> 3
#> 4
#> [ CPUf32{4} ]
```

This will use the `in_tree` and `out_tree` fields of the Graph to flatten and unflatten the list of arrays.

The functions we JIT-compile can also take in so-called “static” arguments, which can be standard R values. Below, we define a function that either multiplies or adds two values, depending on the value of the `op` argument. We need to mark the `op` argument as static.

```
add_or_multiply <- jit(function(x, y, op) {
  if (op == "add") {
    nv_add(x, y)
  } else {
    nv_mul(x, y)
  }
}, static = "op")
```

The difference between static and non-static arguments is that for the latter, the function is recompiled for every unique input type combination (i.e. shape and data type), while for static arguments, the function is recompiled for every unique *value*.

```
add_or_multiply(x, y, "add")

#> AnvilArray
#> 2
#> 3
#> 4
#> 5
#> [ CPUf32{4} ]
```

### Transformations like `gradient()`

Just like `jit()`, the `gradient()` function takes in a function and outputs a function that contains the recipe for transforming the input function into a Graph representing the gradient of the input function.

```
add_fn <- function(x, y) {
  nv_add(x, y)
}
grad <- gradient(add_fn, wrt = c("x", "y"))
```

In order to be able to execute the gradient function, we need to wrap it in a `jit()` call. We can also `jit` a function that contains “standard” anvil operations and calls into the gradient function.

```
x <- nv_scalar(1, "f32")
y <- nv_scalar(5, "f32")
f <- function(x, y) {
  z <- nv_add(x, y)
  grads <- grad(x, z)
  grads
}
```

The way this works is the following: When tracing the function `f`, a `GraphDescriptor` is first initialized. When `nv_add` is called, it populates the `GraphDescriptor` with the inputs `x` and `y` and adds the `PrimitiveCall` to the body representing the addition of `x` and `y`. When we then call into the `grad` function, we create another `GraphDescriptor`, because we need a graph-representation of `add_fn` so we can transform it into its gradient. After this is done, the resulting gradient graph is inlined into the parent `GraphDescriptor`.

We can see the output of this below:

```
graph <- trace_fn(f, list(x = x, y = y))
graph

#> <AnvilGraph>
#> Inputs:
#>   %x1: f32[]
#>   %x2: f32[]
#> Constants:
#>   %c1: f32[]
#> Body:
#>   %1: f32[] = add(%x1, %x2)
#>   %2: f32[] = add(%x1, %1)
#> Outputs:
#>   %c1: f32[]
#>   %c1: f32[]
```

Because this makes `f` just another graph-building function, we can wrap it in a `jit()` call and execute it.

```
f_jit <- jit(f)
f_jit(x, y)

#> $x
#> AnvilArray
#> 1
#> [ CPUf32{} ]
#>
#> $y
#> AnvilArray
#> 1
#> [ CPUf32{} ]
```

### Nested Transformations

Transformations can be nested freely: a gradient function can be called inside a `jit`-compiled function, or a gradient can be taken of a function that itself computes a gradient (to obtain second-order derivatives). This works via *graph inlining*: when a transformation like

`gradient()` is called during tracing, a separate inner graph is traced and transformed, and the result is spliced back into the outer graph. From the outer graph's perspective, the inlined operations are indistinguishable from operations added directly.

## Debugging

One of the main disadvantages of the JIT-compilation approach is that it makes the program harder to debug, because the function cannot simply be executed line-by-line. For this reason, we offer a debugging mode that allows to execute a function without wrapping it in a `jit()` call. There, the function will be executed line-by-line. However, this will mean that we don't execute the real function, but only perform the abstract evaluation, meaning you can also see the types of the intermediate values.

```
multiply_print_r(x, y)
```

```
#> [1] "Hello useR!"
```

```
#> f32{}
```

It is also possible to print intermediate arrays during program execution via `nv_print()`.

```
multiply_print_jit <- jit(function(x, y) {  
  out <- nv_mul(x, y)  
  nv_print(out)  
  return(out)  
})  
multiply_print_jit(x, y)
```

```
#> AnvilArray
```

```
#> 5
```

```
#> [ f32{ } ]
```

```
#> AnvilArray
```

```
#> 5
```

```
#> [ CPUf32{ } ]
```

The difference between `nv_print` and a standard `print()` call is that the former will be embedded in the StableHLO program and executed by the backend, while the latter will be executed during the tracing. The `nv_print()` function is implemented as a StableHLO custom call, cf. the extending section.

Finally, to aid debugging, we try to provide good error messages when applying graph-building functions to the wrong types of inputs.

## Control Flow

Because **anvil** records an evaluation trace, standard R control flow like `if` and `while` is executed at trace time: only the branch that is actually taken during tracing is recorded in the graph, and a `while` loop is unrolled for the number of iterations that happen during tracing. This is sufficient when the control flow depends on static arguments or when the number of loop iterations is known at compile time. However, it does not cover the common case where control flow depends on the values of the arrays being computed, or where the number of iterations should not be fixed at compile time.

For this reason, **anvil** provides two control flow primitives: `nv_while()` and `nv_if()`. These are lowered to StableHLO's native `while` and `if` operations, meaning the control flow becomes part of the compiled program rather than being resolved during tracing. The `nv_while()` function takes an initial state (a named list of arrays), a condition function, and a body function:

```
# Compute x = x * 2 until x > 100
result <- jit(function(x) {
  out <- nv_while(
    list(x = x),
    \(x) x <= nv_scalar(100, "f32"),
    \(x) list(x = nv_mul(x, nv_scalar(2, "f32"))))
  )
  out$x
})
result(nv_scalar(1, "f32"))

#> AnvilArray
#> 128
#> [ CPUf32{} ]
```

This is particularly powerful because it enables users to implement custom iterative algorithms — such as optimization loops, ODE solvers, or MCMC samplers — entirely within a single compiled program, without needing to write custom C++ or CUDA kernels. All operations inside `nv_while()` and `nv_if()` are composed from the same set of [anvil](#) primitives, which means they automatically benefit from the existing StableHLO lowering rules, reverse rules for automatic differentiation, and hardware-agnostic execution via PJRT.

### Random Number Generation

Because [anvil](#) functions must be pure for the code transformation approach to be correct, random number generation cannot rely on global mutable state like R's `.Random.seed`. Instead, [anvil](#) follows an explicit state-passing model: the user creates an RNG state via `nv_rng_state()`, and each random sampling function (e.g., `nv_runif()`, `nv_rnorm()`) takes the current state as input and returns both the random samples and a new state that must be passed to the next call.

```
state <- nv_rng_state(seed = 42L)
result1 <- nv_runif(state, shape = c(3L), dtype = "f32")
state2 <- result1[[1L]]
samples <- result1[[2L]]
```

This design ensures reproducibility and is compatible with JIT compilation, since the RNG state flows through the computational graph like any other value. However, the explicit state threading is more verbose than R's implicit global seed and can be inconvenient for users. We may explore more ergonomic interfaces in the future that reduce this boilerplate while preserving functional purity.

## 4 Examples

Despite [anvil](#) being still in a rather early stage and there are many features we would like to add, it can already be used to accelerate real-world problems. We demonstrate this with a simple logistic regression example that showcases `jit()`, `gradient()`, and `nv_while()` working together.

### 4.1 Logistic Regression via Gradient Descent

To illustrate how `jit()` and `gradient()` work together, we fit a logistic regression model via gradient descent using the Titanic survival dataset from base R. A more detailed version of this example is available as a package vignette.

We first prepare the data by expanding the contingency table into a design matrix and converting it to `AnvilArrays`.

```

set.seed(42)
titanic_df <- as.data.frame(Titanic)
titanic <- titanic_df[rep(seq_len(nrow(titanic_df)), titanic_df$Freq), 1:4]
X <- scale(model.matrix(~ Class + Sex + Age, data = titanic)[, -1])
y <- as.integer(titanic$Survived == "Yes")
n <- nrow(X); p <- ncol(X)

X_array <- nv_array(X, dtype = "f32")
y_array <- nv_array(y, dtype = "f32", shape = c(n, 1L))

```

The model predicts survival probabilities and evaluates them with binary cross-entropy loss:

```

model_loss <- function(X, y, beta, alpha) {
  probs <- nv_logistic(X %*% beta + alpha)
  eps <- 1e-7
  probs <- nv_clamp(eps, probs, 1 - eps)
  loss <- -(y * log(probs) + (1 - y) * log(1 - probs))
  mean(loss)
}

```

We obtain the gradient of the loss with respect to the parameters using `gradient()`:

```
model_loss_grad <- gradient(model_loss, wrt = c("beta", "alpha"))
```

The training loop is implemented using `nv_while()`, which keeps the entire loop within a single compiled function. This avoids the overhead of repeatedly calling into the compiled function from R.

```

fit_logreg <- jit(function(X, y, beta, alpha, n_epochs, lr) {
  output <- nv_while(
    list(beta = beta, alpha = alpha, epoch = nv_scalar(0L)),
    \(beta, alpha, epoch) epoch < n_epochs,
    \(beta, alpha, epoch) {
      grads <- model_loss_grad(X, y, beta, alpha)
      list(
        beta = beta - lr * grads$beta,
        alpha = alpha - lr * grads$alpha,
        epoch = epoch + 1L
      )
    }
  )
  list(beta = output$beta, alpha = output$alpha)
})

```

We initialize the parameters and train the model:

```

beta_init <- nv_array(rnorm(p), dtype = "f32", shape = c(p, 1L))
alpha_init <- nv_scalar(0, dtype = "f32")

result <- fit_logreg(
  X_array, y_array, beta_init, alpha_init,
  nv_scalar(50000L), nv_scalar(0.1)
)

```

To verify correctness, we compare the estimated coefficients with R's built-in `glm()`:

```
#>   Parameter      anvil      glm
#> 1 (Intercept) -0.8538058 -0.8538077
#> 2   Class2nd -0.3418888 -0.3418906
#> 3   Class3rd -0.8299900 -0.8299946
#> 4   ClassCrew -0.4206345 -0.4206313
#> 5   SexFemale  0.9919736  0.9919786
#> 6   AgeAdult -0.2303528 -0.2303616
```

The estimates closely match, confirming that the gradient descent implementation converges to the same solution as `glm()`.

Further examples are available as package vignettes.

## 5 Extending

One of the key design goals of **anvil** is to be extendable. It is possible to add new primitive operations, transformations, and even backends.

### 5.1 New Primitive Operations

The simplest way to extend **anvil** is to add new primitive operations. A detailed guide is available as a package vignette (`vignette("new_primitive", package = "anvil")`). Here we summarize the key steps using the same example from that vignette: `nv1_repeat_along`, a primitive that repeats an array multiple times along a specified dimension. For example, repeating `c(1, 2, 3)` twice along dimension 1 gives `c(1, 2, 3, 1, 2, 3)`.

**Step 1: Create the `AnvilPrimitive` object** that identifies this operation and holds its transformation rules.

```
p_repeat_along <- AnvilPrimitive("repeat_along")
```

**Step 2: Define the `nv1_*` function.** This function defines the type inference rule (how output types are computed from input types) and records the operation in the computational graph via `graph_desc_add()`. The primitive has one *dynamic* input (the array) and two *static* parameters (times and dim).

```
nv1_repeat_along <- function(operand, times, dim) {
  infer_fn <- function(operand, times, dim) {
    if (!checkmate::test_integerish(dim, lower = 1, upper = ndims(operand), len = 1L)) {
      cli::cli_abort("{.arg dim} must be between 1 and {ndims(operand)}, but is {.val dim}")
    }
    if (!checkmate::test_integerish(times, lower = 1, len = 1L)) {
      cli::cli_abort("times must be a positive integer, but is {times}")
    }
    new_shape <- shape(operand)
    new_shape[dim] <- new_shape[dim] * times
    list(AbstractArray(
      dtype = dtype(operand), shape = Shape(new_shape),
      ambiguous = operand$ambiguous
    ))
  }
  graph_desc_add(p_repeat_along, list(operand = operand),
    params = list(times = times, dim = dim), infer_fn = infer_fn)[[1L]]
}
```

**Step 3: Add the `StableHLO` rule.** This defines how to lower the primitive to `StableHLO` operations, implementing `repeat_along` via concatenation. Note the conversion from 1-based to 0-based indexing.



```
p_repeat_along[["stablehlo"]] <- function(operand, times, dim) {
  operands <- rep(list(operand), times)
  list(rlang::exec(stablehlo::hlo_concatenate, !!!operands, dimension = dim - 1L))
}
```

**Step 4: Add the reverse rule.** The reverse rule receives the forward pass inputs and outputs, the incoming gradients, the static parameters by name, and a `.required` flag indicating which input gradients are needed. Here, we reshape the gradient to separate the repeated dimension, then sum over it.

```
p_repeat_along[["reverse"]] <- function(inputs, outputs, grads, dim, times, .required) {
  if (!.required[[1L]]) return(list(NULL))
  grad <- grads[[1L]]
  operand <- inputs[[1L]]
  old_shape <- shape(operand)
  new_shape <- shape(grad)
  new_shape[dim] <- old_shape[dim]
  new_shape <- append(new_shape, times, after = dim - 1L)
  grad_resaped <- nvl_reshape(grad, new_shape)
  list(nvl_reduce_sum(grad_resaped, dims = dim, drop = TRUE))
}
```

**Step 5: Register and use the primitive.**

```
register_primitive("repeat_along", p_repeat_along)
```

With these steps complete, the primitive can be used in JIT-compiled functions and supports automatic differentiation:

```
x <- nv_array(c(1, 2, 3), shape = c(3, 1))
jit(function(x) nvl_repeat_along(x, times = 2L, dim = 2L))(x)

#> AnvilArray
#>  1 1
#>  2 2
#>  3 3
#> [ CPUf32{3,2} ]

jit(gradient(function(x) sum(nvl_repeat_along(x, times = 2L, dim = 2L))))(x)[[1L]]

#> AnvilArray
#>  2
#>  2
#>  2
#> [ CPUf32{3,1} ]
```

Because the backward rule is itself expressed in terms of primitive operations, the gradient computation is built using the same tracing mechanism and can be further transformed or compiled. While this is already quite powerful, there are cases where expressing the operation in terms of existing StableHLO operations is not possible or efficient enough. For this reason, it is in principle possible to implement custom calls that perform arbitrary operations defined in C++, possibly using CUDA. This is not super ergonomic yet, but this could be improved if it shows to be necessary.

## 5.2 New Transformations

Because the Graph representation is a simple data structure consisting of inputs, outputs, and a sequence of `PrimitiveCall` objects, it is straightforward to implement new graph-to-graph transformations. The `transform_gradient()` function, which implements backward-mode automatic differentiation, is the primary example of such a transformation. It walks the graph in reverse order and applies the reverse rule stored in each primitive's `@rules[["reverse"]]` field to accumulate gradients.

Implementing a new transformation follows the same pattern: iterate over the `PrimitiveCall` objects in the graph, look up a corresponding rule for each primitive, and use these rules to construct a new graph. For example, one could implement forward-mode automatic differentiation by walking the graph *forward* and propagating Jacobian-vector products, or implement a graph optimization pass that fuses or simplifies sequences of primitive calls. The key design decision is that the `anvil` package does not impose any constraints on how a transformation is implemented — it only provides the `Primitive` rule mechanism as a convenient way to store per-operation information.

## 5.3 Custom Backends

Adding a new backend amounts to implementing a new graph-to-executable transformation, analogous to how the XLA backend translates a Graph into a StableHLO program via the `stablehlo()` function and then compiles it via PJRT. For each primitive, a lowering rule is stored in the `@rules` field (e.g., under "stablehlo" for the XLA backend). A new backend would define its own set of rules under a different name and provide a corresponding lowering function that walks the graph and applies these rules to produce the target representation.

As a concrete example, there is an experimental backend that compiles Graph objects to Fortran code via the `quickr` package, targeting CPU execution without requiring the XLA toolchain. This demonstrates that the same graph can be lowered to fundamentally different targets. However, the process of adding a new backend currently requires some familiarity with the package internals. Providing a more streamlined and well-documented interface for registering custom backends is an area we plan to improve.

## 6 Testing

Ensuring correctness of both the forward evaluation and the gradient computation is critical for a code transformation framework. In `anvil`, we bootstrap correctness by comparing against the `torch` package wherever possible.

For forward-pass testing, each primitive's JIT-compiled output is compared against the corresponding `torch` function on randomly generated inputs. For example, to test `nv1_add`, we generate random arrays, compute the result using both `jit(nv1_add)(x, y)` and `torch::torch_add(x, y)`, and assert that the outputs match up to numerical tolerance. This is done using helper functions like `expect_jit_torch_binary()` and `expect_jit_torch_unary()`, which handle the data generation and comparison.

For backward-pass testing, the same strategy is applied to gradient computation. The `anvil` gradient (obtained via `jit(gradient(f))(x)`) is compared against the gradient computed by `torch`'s autograd engine. For non-scalar functions, both sides are wrapped with a summation to produce a scalar loss before differentiating, ensuring a consistent comparison.

Because `torch` is not a hard dependency of the package, the `torch`-based tests are stored in `inst/extra-tests/` and are conditionally sourced only when `torch` is installed. This keeps the test suite lightweight for users who do not have `torch`, while still running the full comparison suite in CI. For primitives that have no direct `torch` equivalent (e.g., random number generation or bitwise operations), manual tests with hardcoded expected values are used instead.

## 7 Benchmarks

Runtime benchmarks comparing **anvil** on CPU and GPU are available at <https://r-xla.github.io/benchmarks/>.

## 8 Discussion

In this paper, we introduced the **anvil** package, which is a code transformation framework for the R language. Execution is handled via JIT compilation through either the OpenXLA compiler framework or by transpiling to Fortran via **quicksr**. When using the OpenXLA backend, it is possible to make use of accelerators such as GPUs. Furthermore, the package supports first-order reverse-mode automatic differentiation.

### 8.1 Key Advantages

Several design properties distinguish **anvil** from the other R packages discussed in the related work section.

First, the code transformation engine is implemented almost entirely in pure R. Unlike **torch**, which wraps the large LibTorch C++ library, or **tensorflow** and **keras3**, which require a Python runtime via **reticulate**, **anvil**'s frontend has no Python dependency and requires no C++ compilation. This makes the package easier to install, easier to contribute to, and easier to debug — the entire transformation pipeline is readable R code. Internally, the package relies on S3 for its object system. We initially attempted to use the newer S7 system (Vaughan et al., 2025), which would have been a natural fit given the package's reliance on method dispatch for different node and type classes. However, the runtime overhead introduced by S7's dynamic property validation made it prohibitively slow for a framework in which objects are created and dispatched on at very high frequency during tracing and compilation.

Second, the composability of transformations is a central design principle. Because each transformation operates on a self-contained Graph that is spliced back into its parent context, transformations can be nested freely: a gradient can be taken of a function that itself computes a gradient, or a gradient function can be called inside a jit-compiled function. To our knowledge, no other native R package offers this capability.

Third, by offering only JIT compilation (with a separate debug mode for type checking), **anvil** avoids the class of bugs that arise in frameworks with dual eager/JIT execution modes, where code that works in one mode can silently produce different results in the other.

Fourth, because **stablehlo** and **pjrt** are independent packages, they can be used outside of **anvil**. StableHLO programs generated by other frameworks can be executed in R via **pjrt**, and R-generated StableHLO can be consumed by any tool in the OpenXLA ecosystem.

Finally, the rule-based extensibility model means that new primitives — including their StableHLO lowering and reverse-mode differentiation rules — can be added entirely in R, without requiring any C++ or CUDA code.

### 8.2 Limitations

The biggest limitations of the current approach are twofold:

- a) Programs need to have static shapes, which means that operations whose output shape depends on the input values — such as `unique()` or `which()` — are currently not supported.
- b) For the OpenXLA backend, programs need to be re-compiled for each new unique input type combination (i.e., each unique combination of shapes and data types).

### 8.3 Future Work

Future work will aim at lifting the static shape and re-compilation restrictions. One solution will be to improve the support for the Fortran backend, which — as opposed to XLA — does support shape dynamism. To do this properly, the intermediate Graph representation needs to be extended to support shape dynamism, which it currently does not. Extending the Fortran backend to cover more primitives also means that the re-compilation restriction can be lifted, at least when executing on CPUs.

Another important improvement is to decouple the intermediate representation from StableHLO more thoroughly. Currently, the IR is tied too closely to StableHLO, which limits the ability to target alternative backends and to represent higher-level optimizations that do not map directly to StableHLO concepts.

Beyond these architectural improvements, there are many features that would increase the usefulness of the package. Extending the set of primitive operations in both `anvil` and `stablehlo` would enable more API functions, particularly for linear algebra. Additional code transformations — such as forward-mode AD, higher-order derivatives (e.g. by truncating taylor polynomials CITE), and batching transformations — would broaden the scope of the framework. For example, forward mode AD can be used for delta approximation. Building a small modeling library on top of `anvil` for gradient-based optimization would further demonstrate its practical utility.

Usability of the package would also benefit from streamlining the installation process. In order to use GPUs with the XLA backend, manual effort is currently required to install the specific dependencies (CUDA, cuDNN, NVSHMEM), which should be simplified.

## 9 Acknowledgments

Sebastian Fischer is supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 460135501 (NFDI project MaRDI).

## 10 Appendix

### 10.1 AI Usage

AI coding agents were used for the development of the software. All generated code was carefully checked by the authors.

## References

- B. M. Bell. Cppad: a package for c++ algorithmic differentiation. *Computational Infrastructure for Operations Research*, 57(10):3, 2012. [p3]
- J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. Jax: Composable transformations of python+numpy programs. Github repository. URL <https://github.com/google/jax>, 2018. [p1, 2, 10]
- P.-C. Bürkner. brms: An r package for bayesian multilevel models using stan. *Journal of statistical software*, 80:1–28, 2017. [p3]
- B. Carpenter, A. Gelman, M. D. Hoffman, D. Lee, B. Goodrich, M. Betancourt, M. Brubaker, J. Guo, P. Li, and A. Riddell. Stan: A probabilistic programming language. *Journal of statistical software*, 76:1–32, 2017. [p3]
- D. Eddelbuettel. Seamless r and c++ integration with rcpp. 2013. [p5]

- D. Falbel. *safetensors: Safetensors File Format*, 2025. URL <https://CRAN.R-project.org/package=safetensors>. R package version 0.2.0. [p7]
- D. Falbel and J. Luraschi. *torch: Tensors and Neural Networks with 'GPU' Acceleration*, 2025. URL <https://CRAN.R-project.org/package=torch>. R package version 0.16.3. [p2]
- A. Griewank and A. Walther. *Evaluating derivatives: principles and techniques of algorithmic differentiation*. SIAM, 2nd edition, 2008. doi: 10.1137/1.9780898717761. [p1, 4, 8, 9]
- C. R. Harris, K. J. Millman, S. J. Van Der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020. [p2]
- T. Kalinowski. *quicksr: Compiler for R*, 2026. URL <https://github.com/t-kalinowski/quicksr>. R package version 0.2.1.9000, commit d38a27377e3c1b43ddaa22ba234ffdbface2454. [p3]
- F. X. Konrad Krämer. *ast2ast*, 2026. URL <https://github.com/Konrad1991/ast2ast/>. [p3]
- K. Kristensen, A. Nielsen, C. W. Berg, H. Skaug, and B. M. Bell. Tmb: Automatic differentiation and laplace approximation. *Journal of Statistical Software*, 70(5), 2016. ISSN 1548-7660. doi: 10.18637/jss.v070.i05. URL <http://dx.doi.org/10.18637/jss.v070.i05>. [p3]
- F. Morandat, B. Hill, L. Osvald, and J. Vitek. Evaluating the design of the r language: Objects and functions for data analysis. In *European conference on object-oriented programming*, pages 104–131. Springer, 2012. [p3]
- A. Munshi. The opencl specification. In *2009 IEEE Hot Chips 21 Symposium (HCS)*, pages 1–314. IEEE, 2009. [p3]
- OpenXLA Team. *OpenXLA*, 2026. URL <https://openxla.org/>. [p1]
- A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019. [p2]
- S. E. Pav. *madness: Automatic Differentiation of Multivariate Operations*, 2023. URL <https://github.com/shabbychef/madness>. R package version 0.2.8. [p3]
- H. Perdry. *salad: Simple Automatic Differentiation*, 2024. URL <https://CRAN.R-project.org/package=salad>. R package version 1.2. [p3]
- J. Pfeifer. *gpjrt: Go Wrappers for OpenXLA PJRT*, 2026. URL <https://github.com/gomlx/gpjrt>. [p2]
- PJRT Team. *PJRT: Pluggable JIT RunTime*, 2026. URL <https://openxla.org/pjrt/>. [p1, 4]
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2025. URL <https://www.R-project.org/>. [p3]
- L. Sartore. *dual: Automatic Differentiation with Dual Numbers*, 2025. URL <https://CRAN.R-project.org/package=dual>. R package version 0.0.6. [p3]
- StableHLO Team. *StableHLO*, 2026. URL <https://openxla.org/stablehlo/spec>. [p1, 4]
- G. Team. *Go Software, Version 1.25*, 2025. URL <https://go.dev/>. [p2]
- The JAX Authors. Autodidax: JAX core from scratch, 2024. URL <https://docs.jax.dev/en/latest/autodidax.html>. [p2]
- The MicroJAX Authors. *MicroJAX*, 2024. URL <https://github.com/joey0072/microjax>. [p2]
- A. Towell. *nabla: Exact Derivatives via Automatic Differentiation*, 2026. URL <https://github.com/queelius/nabla>. R package version 0.7.1. [p3]

D. Vaughan, J. Hester, T. Kalinowski, W. Landau, M. Lawrence, M. Maechler, L. Tierney, and H. Wickham. *S7: An Object Oriented System Meant to Become a Successor to S3 and S4*, 2025. URL <https://CRAN.R-project.org/package=S7>. R package version 0.2.1. [p23]

*Sebastian Fischer*  
LMU MunichMunich Center for Machine Learning  
Department of Statistics  
Ludwig-Maximilians-Universität Munich  
Ludwigstr. 33, 80539 Munich, Germany  
ORCID: [0000-0002-9609-3197](https://orcid.org/0000-0002-9609-3197)  
[sebastian.fischer@stat.uni-muenchen.de](mailto:sebastian.fischer@stat.uni-muenchen.de)

*Daniel Falbel*  
Posit Software, PBC

ORCID: [0000-0002-0912-0225](https://orcid.org/0000-0002-0912-0225)

*Thomas Kalinowski*  
Posit Software, PBC

*Niko German*  
LMU Munich

ORCID: [0009-0001-7394-8367](https://orcid.org/0009-0001-7394-8367)

*Martin Binder*  
LMU MunichMunich Center for Machine Learning

ORCID: [0009-0008-2578-2869](https://orcid.org/0009-0008-2578-2869)

*Bernd Bischl*  
LMU MunichMunich Center for Machine Learning

ORCID: [0000-0001-6002-6980](https://orcid.org/0000-0001-6002-6980)